PQC-Secure Client-Server System

Post-Quantum Cryptography Protocol — Technical Report

F. Zehra ATES & Zeynep OZCELIK | June 2026

1. Introduction

The growth of quantum computing presents a serious and growing threat to the security of modern communication systems. Many of the cryptographic algorithms used today — such as RSA, ECDH, and ECDSA — rely on problems that are considered very hard for classical computers to solve, such as factoring large integers or computing discrete logarithms [12]. However, these same problems can be solved efficiently by a sufficiently powerful quantum computer running Shor's algorithm [15]. This means that a future adversary with a powerful quantum computer could retroactively break the encryption protecting data that was transmitted years earlier, even if that data was captured and stored long before any quantum computer existed. This risk is commonly known as the "harvest now, decrypt later" attack. Even symmetric algorithms such as AES are weakened by Grover's algorithm [16], though AES-256 retains an effective security level of 128 bits against quantum adversaries.

In response to this threat, the National Institute of Standards and Technology (NIST) completed a multi-year standardization effort [7] and in 2024 published its first post-quantum cryptography (PQC) standards. These include ML-KEM (also called CRYSTALS-Kyber) for key encapsulation mechanisms [5], and ML-DSA (also called CRYSTALS-Dilithium) for digital signatures [6]. Both algorithms are based on mathematical problems involving module lattices, for which no efficient quantum algorithm is currently known [8][9]. These standards mark the beginning of a practical transition from classical to post-quantum cryptographic infrastructure.

This report documents the full design, implementation, and evaluation of a post-quantum secure client-server communication system. The system is built around a custom handshake and record layer protocol that is closely inspired by TLS 1.3, and it supports three distinct operating modes, allowing direct performance and security comparisons:

- PQ_ONLY: Provides full post-quantum security using Kyber-768 (ML-KEM-768) for key encapsulation and Dilithium-3 (ML-DSA-65) for digital signatures. This mode is designed for production environments that require full quantum resistance today.
- CLASSICAL: Uses X25519 for key exchange and Ed25519 for digital signatures. This mode offers no quantum resistance, but it serves as an important performance baseline for comparing the cost of post-quantum primitives.
- HYBRID: Combines X25519 and Kyber-768 for key exchange, so that the security of the channel holds as long as at least one of the two algorithms remains unbroken. This mode is recommended for the transitional period, when organizations are moving from classical to post-quantum cryptography but want to maintain backward compatibility and defense in depth.

All three modes share the same record layer (AES-256-GCM), the same handshake state machine, and the same certificate infrastructure. Only the underlying cryptographic providers differ. This design makes it easy to compare the performance impact of each approach while keeping all other variables constant.

The primary contributions of this work are:

- A fully implemented post-quantum handshake protocol with canonical binary encoding, transcript binding, certificate validation, identity binding, and stateless DoS mitigation via HMAC cookies.
- A record layer that provides AES-256-GCM authenticated encryption, with replay protection using an IPsec-style sliding window, automatic rekeying via HKDF, and strict fail-closed error handling.
- Quantitative performance measurements for all three modes, including handshake latency across 100 trials, record layer throughput across six payload sizes, and direct comparisons between Dilithium2/Dilithium3 and Kyber512/Kyber768 parameter sets.

- A security evaluation covering six attack types (replay, bit-flip, fake server, downgrade, eavesdrop, and MITM) across all three modes, with 100% detection rate and zero false positives across 18 configurations.
- A comprehensive test suite with 34 passing tests across 10 modules, covering both happy-path operation and adversarial scenarios.

2. System Overview

The system creates a secure communication channel between a client and a server using a custom protocol modeled after TLS 1.3. The core design principle is a clean separation between the cryptographic negotiation phase (handshake layer) and the data protection phase (record layer), so that each component can be independently analyzed, tested, and replaced without affecting the other.

2.1 Purpose and Architecture

The system's main goal is to establish a mutually authenticated, confidential, and integrity-protected communication channel that remains secure against adversaries with access to both classical and quantum computers. Both the client and the server present digital certificates signed by a shared Certificate Authority (CA). The session keys used to protect application data are derived from a fresh shared secret that is generated independently for each connection, ensuring forward secrecy.

The architecture uses TCP as the transport layer. The server listens at a fixed address (127.0.0.1:33354 by default) and handles one connection per thread. Each incoming connection must complete the full handshake phase before any application data is exchanged. The system is structured as four vertically stacked layers, each with a single, well-defined responsibility:

Layer	Directory	Responsibility
Application	client.py / server.py	Entry points, argument parsing, and mode selection
Handshake	protocol/	Message encoding, transcript, state machines, provider factory
Record	record/	AES-256-GCM encryption, key schedule, replay window, rekeying
Transport	network/	TCP framing (4-byte length prefix), send/receive wrappers

The following diagram shows the overall system architecture, illustrating how the client and server communicate across the four layers, and how the shared security/ module interacts with the handshake layer on both sides.

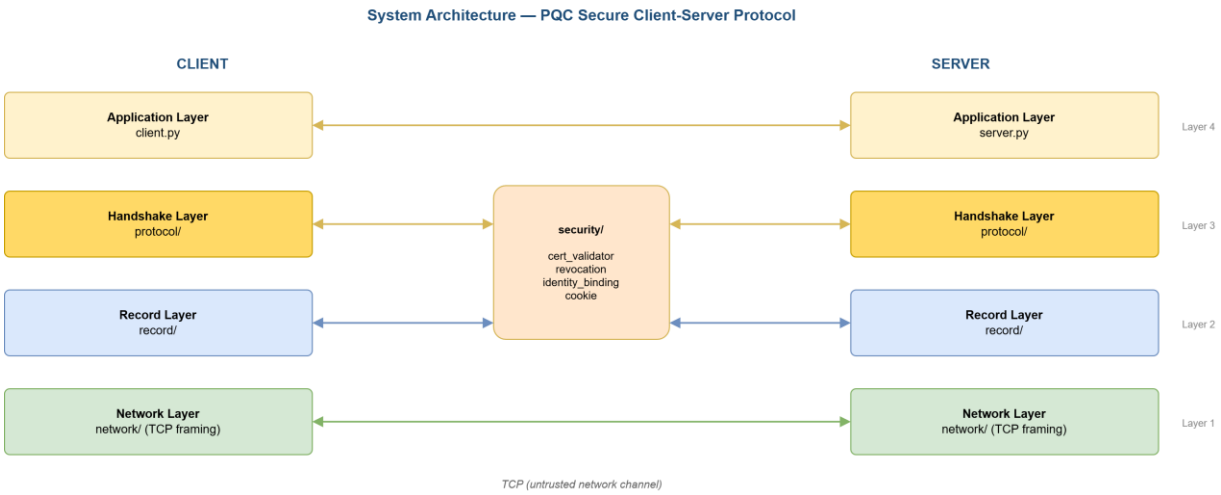


Figure 2.1: *System Architecture — PQC Secure Client-Server Protocol. Both client and server share the same four-layer stack. The security/ module (cert_validator, revocation, identity_binding, cookie) operates between the handshake layer and the network, enforcing all certificate and DoS-mitigation checks before any key material is established.*

2.2 Handshake and Record Layer Separation

The handshake layer (implemented in the protocol/ directory) is responsible for mutual authentication, algorithm negotiation, and session key establishment. When the handshake completes successfully, it produces four directional session keys — k_cs, k_sc, iv_cs, and iv_sc — which are passed directly to the record layer. The handshake layer never processes application data.

The record layer (implemented in the record/ directory) uses these session keys to provide AES-256-GCM authenticated encryption for every message exchanged after the handshake completes. The two layers are cleanly isolated from each other: the record layer has no knowledge of certificates, KEM operations, or digital signatures. It only receives key material from the handshake, and it applies that key material uniformly regardless of which operating mode was used. This separation also means that the record layer code does not need to change when a new handshake mode is added.

2.3 Supported Modes

Mode	Key Exchange	Authentication	Quantum Resistant	Primary Use
PQ_ONLY	Kyber-768 (ML-KEM-768)	Dilithium-3 (ML-DSA-65)	Yes — fully	Production PQ deployment
CLASSICAL	X25519	Ed25519	No	Performance baseline
HYBRID	X25519 + Kyber-768	Dilithium-3 (ML-DSA-65)	Yes — if either holds	Transitional deployment

The operating mode is selected at startup via the --mode command-line argument and affects only the cryptographic providers used in the handshake. All three modes share the same record layer (AES-256-GCM), the same handshake state machine, and the same certificate infrastructure, making it straightforward to run controlled performance comparisons.

2.4 Trust Model

Both the client and the server possess certificates that were signed by a single shared Certificate Authority (CA). The CA's public key is pre-distributed and pinned at both endpoints. During the handshake, each party independently verifies the other's certificate against this pinned CA key, checking the digital signature, the validity dates, and the identity binding. This is a controlled trust model that is appropriate for a research prototype and for controlled deployments where both endpoints are managed by the same organization. A production deployment in an open environment would require a full PKI hierarchy, with multiple intermediate CAs, certificate revocation infrastructure, and hardware security module (HSM) protection for CA private keys.

3. Final Protocol Design

3.1 Protocol Architecture

The trust model relies on a single shared CA whose public key is embedded in both the client and the server. Every certificate is a JSON structure containing an identity field, a public key, validity dates (valid_from and valid_to), a serial number for revocation purposes, and a Dilithium signature produced by the CA. Importantly, the CA always uses Dilithium for signing, regardless of the operating mode in use. This design ensures that post-quantum authenticity is maintained even in CLASSICAL mode, because the certificates themselves are always protected by a quantum-resistant signature algorithm.

3.2 Handshake Protocol

The handshake consists of seven ordered steps. Each message is encoded in a canonical binary format using the wire_codec module and appended to a running transcript hash before being signed or used in key derivation. Canonical binary encoding (rather than JSON) is used to ensure that the transcript is completely unambiguous — there is no possibility of two different byte sequences representing the same logical message, which could otherwise lead to transcript manipulation attacks.

The complete message flow is illustrated in the sequence diagram below:

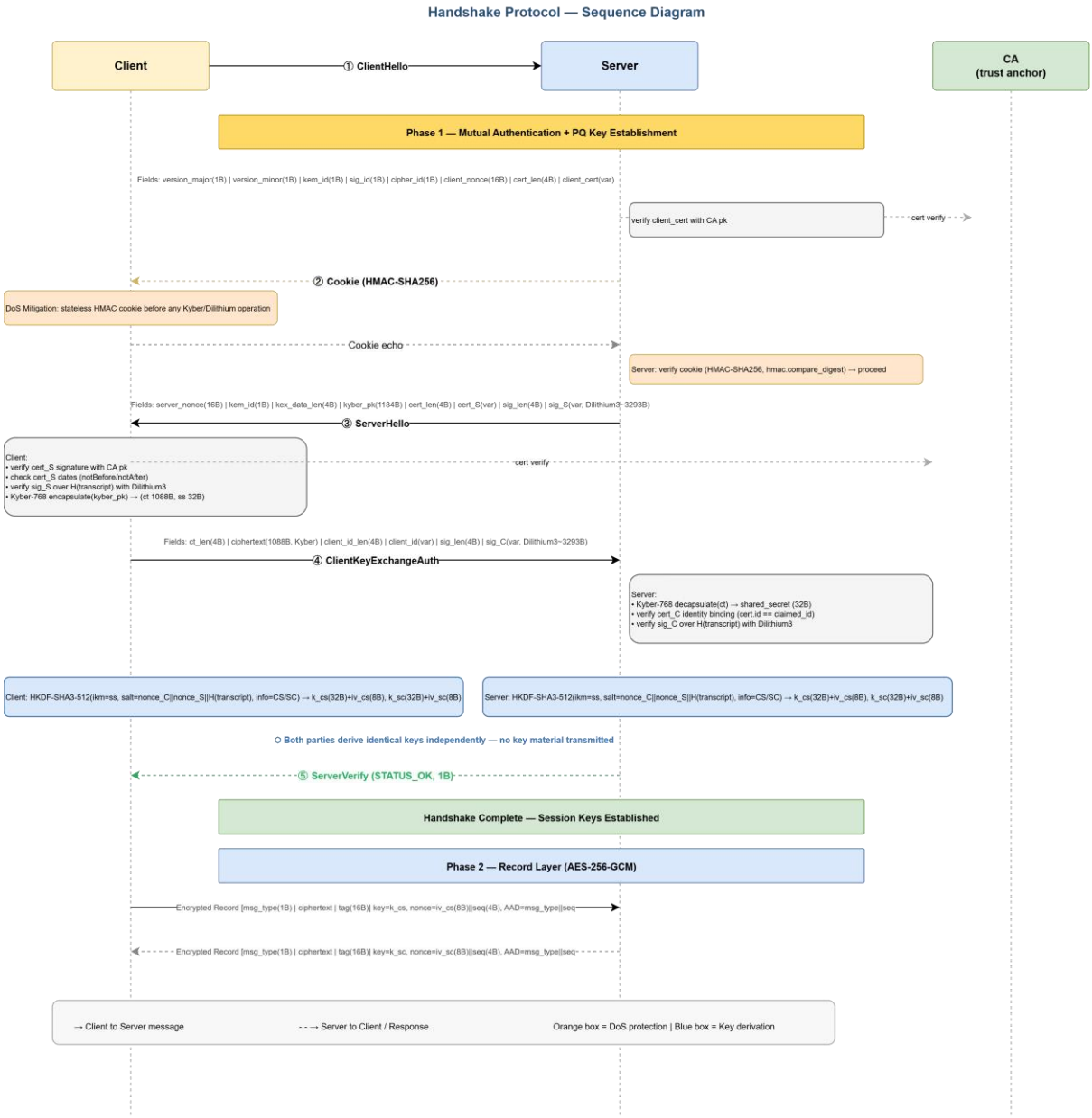


Figure 3.1: *Handshake Protocol Sequence Diagram. Phase 1 covers mutual authentication and PQ key establishment through seven ordered steps. Phase 2 shows the record layer operating with the derived AES-256-GCM session keys after the handshake completes. The CA trust anchor is shown separately on the right, as it participates only through pre-distributed public key pinning, not through any online interaction.*

Step 1 — ClientHello

The client generates a fresh 16-byte random nonce (client_nonce) and encodes it together with its certificate and the protocol version fields (major=1, minor=0) into a ClientHello frame. The frame is transmitted to the server using a 4-byte length-prefixed framing scheme. The encoded bytes are stored for later inclusion in the transcript. The protocol version, KEM identifier, signature identifier, and cipher identifier fields allow the server to detect and reject any downgrade attempts.

Field	Size	Description
version_major	1 byte	Protocol version major (0x01)
version_minor	1 byte	Protocol version minor (0x00)

kem_id	1 byte	Proposed KEM algorithm (0x01=Kyber768, 0x02=X25519, 0x03=Hybrid)
sig_id	1 byte	Proposed signature algorithm (0x01=ML-DSA-65, 0x02=Ed25519)
cipher_id	1 byte	Proposed AEAD cipher (0x01=AES-256-GCM)
client_nonce	16 bytes	Random session nonce generated by the client
cert_len	4 bytes	Length of client certificate (big-endian)
client_cert	variable	JSON-encoded client certificate with CA signature

Step 2 — Cookie Exchange (DoS Mitigation)

Before performing any expensive cryptographic operation, the server generates a stateless HMAC-SHA256 cookie keyed on the client's IP address and port number. This cookie is sent to the client as a raw frame, without any cryptographic computation on the server side. The client must echo the cookie back unchanged. The server then verifies the echo using hmac.compare_digest() for constant-time comparison, which prevents timing-based side-channel attacks on the cookie comparison itself. Only if this verification succeeds does the server proceed to the computationally expensive ServerHello construction.

This mechanism prevents resource exhaustion attacks: an attacker who sends a flood of unauthenticated ClientHello messages will only cause the server to compute cheap HMAC operations, not Dilithium signatures or Kyber key pairs. The server maintains no per-client state before cookie verification completes.

```
cookie = HMAC-SHA256(key=server_secret, msg=client_ip || client_port)
```

Step 3 — ServerHello

After the cookie is verified, the server performs the following operations and encodes the results into a ServerHello frame:

- Generates a fresh 16-byte server_nonce.
- Generates a Kyber-768 keypair (or an X25519 keypair in CLASSICAL mode, or both in HYBRID mode). The public key is included in the ServerHello.
- Computes a Dilithium-3 signature over the SHA3-512 hash of the transcript at this point (which contains only the ClientHello bytes). This signature proves that the server has not been replaced by an impersonator.
- Encodes the server certificate, server nonce, KEM public key(s), and signature into the binary ServerHello frame.
- Sends the frame to the client and appends it to the transcript.

Field	Size	Description
server_nonce	16 bytes	Random session nonce generated by the server
kem_id	1 byte	Confirmed KEM algorithm
sig_id	1 byte	Confirmed signature algorithm
kex_data_len	4 bytes	Length of key exchange data blob
kex_data	variable	TLV-encoded KEM public key(s)
cert_len	4 bytes	Length of server certificate

server_cert	variable	JSON-encoded server certificate
sig_len	4 bytes	Length of Dilithium signature
signature	variable	Dilithium-3 signature over the current transcript hash

Step 4 — Certificate Verification and KEM Encapsulation

Upon receiving the ServerHello, the client performs a sequence of security checks before using any of the server's key material:

- Decodes the server certificate from its JSON encoding and verifies the CA's Dilithium signature using verify_and_validate().
- Checks the certificate's validity dates: valid_from must be in the past and valid_to must be in the future relative to the current UTC time.
- Verifies the Dilithium-3 signature over the current transcript hash using the server's public key extracted from the certificate. This proves the ServerHello was produced by a party that holds the server's private key.
- Performs KEM encapsulation on the server's public key to produce a ciphertext (ct) and a shared secret (ss). The shared secret is never transmitted; the server will independently derive it by decapsulating the ciphertext.

If any of these checks fail, the client immediately raises a ValueError and closes the TCP connection. No partial state is retained, and no key material is derived from a failed handshake.

Step 5 — ClientKeyExchangeAuth

The client encodes the KEM ciphertext and its claimed client_id into an unsigned data blob, appends this blob to the transcript, computes the new transcript hash, and signs the hash with its own Dilithium-3 private key. The resulting signed frame is sent to the server. This step simultaneously achieves two goals: it authenticates the client (proving ownership of the client's private key), and it cryptographically binds the KEM ciphertext to the complete handshake transcript. This binding prevents an attacker from substituting a different ciphertext without the signature check failing.

Step 6 — Key Derivation

Both parties independently derive the session keys as soon as they each have the shared secret. The client already has the shared secret from KEM encapsulation; the server will obtain it by decapsulating the ciphertext in Step 7. The key schedule used for derivation is described in full detail in Section 3.4. The transcript at this point contains the concatenation of: ClientHello bytes || ServerHello bytes || ClientKeyExchangeAuth bytes. The SHA3-512 hash of this complete transcript is included in the HKDF salt, which cryptographically binds the derived session keys to every message exchanged in the handshake.

Step 7 — ServerVerify

The server performs the final verification steps and confirms handshake completion:

- Verifies the client's certificate: checks the CA signature, the validity dates, and the identity binding (the client_id in the certificate must match the claimed client_id in the ClientKeyExchangeAuth message).
- Decapsulates the KEM ciphertext using its private key to obtain the shared secret.
- Derives the same set of session keys using the same key schedule, independently from the client.

- Sends a 1-byte STATUS_OK frame to the client to confirm that the handshake completed successfully on the server side.

The client verifies the STATUS_OK frame, and both parties transition to the record layer. If the key derivation produced matching keys (which is guaranteed by the transcript binding if neither party was tampered with), all subsequent records will decrypt successfully.

Security Properties of the Handshake

> Canonical binary framing: Fixed-width binary encoding eliminates all transcript ambiguity.
> Transcript hash binding: Any modification to any message changes the hash, causing key mismatch.
> Protocol version enforcement: Server rejects unknown version fields immediately.
> Certificate validation: CA signature, validity dates, and identity are all checked before key use.
> Cookie-based DoS mitigation: No expensive operations before the cookie round-trip succeeds.
> Identity binding: Certificate client_id must match the claimed client_id in ClientKeyExchangeAuth.

3.3 Record Protocol

The record layer provides confidentiality, integrity, and authenticity for all application data exchanged after the handshake. It is implemented in record/record_layer.py through the RecordCipher class. The record layer uses AES-256-GCM with replay protection, automatic rekeying, and strict fail-closed error handling. Every record is independently protected, which means that even if an attacker can delay or reorder packets at the network level, the record layer will detect and reject any such manipulation.

3.3.1 Encryption Algorithm

Each record is independently encrypted using AES-256-GCM (Galois/Counter Mode with a 256-bit key). AES-256-GCM is an Authenticated Encryption with Associated Data (AEAD) scheme that provides both confidentiality — through CTR-mode encryption — and integrity and authenticity — through a 128-bit Galois authentication tag. Any modification to the ciphertext, the associated data, or the authentication tag causes the decryption to fail immediately. The scheme is standardized in NIST SP 800-38D [13] and is widely used in TLS 1.3, QUIC, and other modern protocols.

3.3.2 Directional Keys

The handshake key schedule produces four values: k_cs (32 bytes), k_sc (32 bytes), iv_cs (8 bytes), and iv_sc (8 bytes). Client-to-server traffic is encrypted with k_cs and the nonce base iv_cs; server-to-client traffic is encrypted with k_sc and iv_sc. This directional isolation ensures that compromise of one direction's key material does not affect the other direction. It also prevents reflection attacks, where an attacker might attempt to replay a record from one direction into the other.

3.3.3 Nonce Construction

The 12-byte AES-GCM nonce for each record is constructed as follows:

```
nonce = iv_base (8 bytes) || seq_num (4 bytes, big-endian)
```

Each record increments seq_num before encryption, guaranteeing that the nonce is unique within a session and within a direction. Nonce reuse under AES-GCM would be catastrophic: it would allow an attacker to recover the XOR of two plaintexts and would completely break the authentication tag, effectively destroying both confidentiality and integrity for all affected records.

3.3.4 Associated Data (AAD)

A 5-byte AAD value is constructed for each record and authenticated (but not encrypted) by AES-256-GCM:

```
AAD = msg_type (1 byte) || seq_num (4 bytes, big-endian)
```

Including msg_type in the AAD prevents type confusion attacks, where a record of one type might be interpreted as a different type. Including seq_num prevents sequence position substitution, where an attacker might attempt to replay a legitimate record at a different position in the sequence. Both msg_type and seq_num must match the expected values for decryption to succeed.

3.3.5 Replay Protection

An IPsec-style sliding window (size 64, implemented in record/replay_window.py) tracks received sequence numbers for each direction independently using a bitmask. Before any decryption is attempted, seq_num is validated against the window:

- If seq > max_seq: the record is accepted and the window advances (new sequence).
- If seq < max_seq - WINDOW_SIZE: the record is rejected as "too old" (Fail-Closed, ValueError raised).
- If seq is within the window and has already been seen: the record is rejected as a replay (Fail-Closed).
- If seq is within the window and has not been seen: the record is accepted and marked as seen.

The following diagram illustrates the sliding window mechanism and the validation flow for each incoming sequence number:

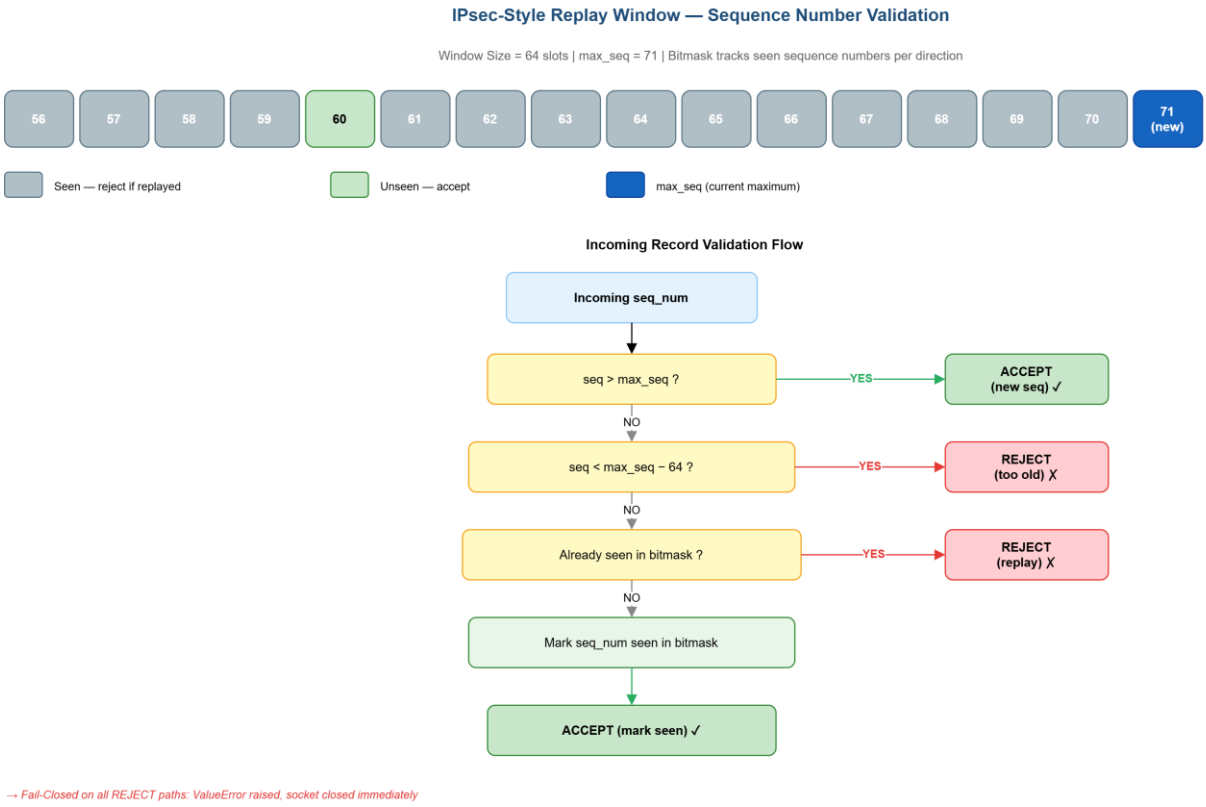


Figure 3.2: *IPsec-Style Replay Window — Sequence Number Validation. The window of 64 slots slides forward as new sequence numbers are accepted. Sequence numbers that fall behind the window are*

rejected as "too old"; sequence numbers within the window that have already been seen are rejected as replays. All rejections immediately close the connection.

3.3.6 Rekeying

After every REKEY_AFTER_RECORDS records (10,000 by default), new key material is derived from the current key and IV base using HKDF-SHA3-512. Rekeying limits the amount of data protected under any single key, which reduces the risk of nonce exhaustion and limits the window of data that would be affected if a key were ever compromised.

```
okm     = HKDF-SHA3-512(ikm=old_key, salt=old_iv, info=b'rekey'||seq, len=40)
new_key = okm[:32]    # new AES-256 key
new_iv  = okm[32:40]  # new 8-byte IV base
```

The RecordCipher instance is updated atomically: the seq_send and seq_rcv counters continue from where they left off, so no records are lost or duplicated during the rekey transition. The application layer is not involved in or notified of rekeying events.

3.3.7 Fail-Closed Behavior

All security failures — including AES-GCM tag verification failure, replay window rejection, unknown msg_type, and payload too short to contain a valid tag — raise a ValueError. The record_flow._fail_closed() function first closes the TCP socket, then raises the exception. This ensures that the connection is always terminated immediately on any security violation, before any partial plaintext could reach the application. The application layer receives either a complete, verified plaintext record or an exception indicating that the connection has been closed.

3.3.8 Record Layer Sequence Diagram

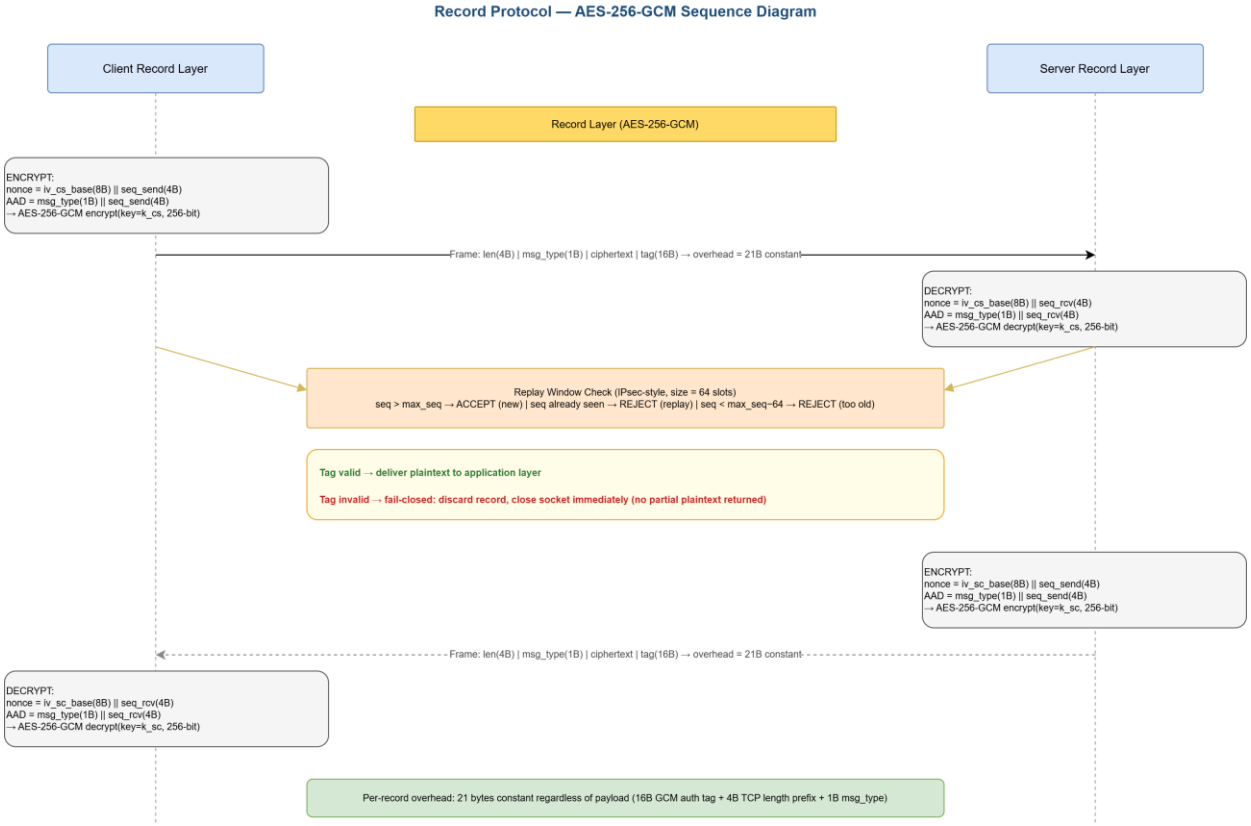


Figure 3.3: *Record Protocol — AES-256-GCM Sequence Diagram. Each direction uses independent keys (k_cs/iv_cs for client-to-server, k_sc/iv_sc for server-to-client). The replay window check occurs before decryption. Invalid tags result in immediate connection termination (fail-closed). Per-record overhead is a constant 21 bytes: 16 B GCM tag + 4 B length prefix + 1 B msg_type.*

3.3.9 Record Layer Summary

Property	Mechanism	Implementation
Confidentiality	AES-256-GCM encryption	RecordCipher.record_encrypt()
Integrity	128-bit GCM authentication tag	AESGCM256.decrypt()
Nonce uniqueness	iv_base || seq_num	RecordCipher._make_nonce()
AAD binding	msg_type || seq_num	RecordCipher._make_aad()
Replay protection	IPsec sliding window (64)	ReplayWindow
Key freshness	HKDF rekeying every N records	rekey.py
Fail-closed	Socket closed on any failure	record_flow._fail_closed()
Directional isolation	Separate keys per direction	key_schedule.derive_session_keys()

3.4 Key Schedule

The key schedule transforms the KEM-derived shared secret into the four directional session keys used by the record layer. It is implemented in record/key_schedule.py and uses the HKDF implementation in crypto/kdf.py.

3.4.1 Salt Construction

Before HKDF is called, a 96-byte salt is assembled from three components:

```
salt = client_nonce (16 bytes) || server_nonce (16 bytes) || transcript_hash (64 bytes)
```

The transcript_hash is the SHA3-512 hash of all handshake messages concatenated in order. Including it in the HKDF salt ensures that the derived session keys are cryptographically bound to the complete handshake transcript. If any handshake message was tampered with, the transcript hash will differ between the two parties, and they will derive different keys, causing all subsequent records to fail to decrypt. This is the cryptographic guarantee that makes transcript binding effective.

3.4.2 HKDF Derivation

HKDF-SHA3-512 is called twice [14], using different info parameters for each direction, to produce the four session key components:

```
okm_cs = HKDF(ikm=shared_secret, salt=salt, info=KDF_INFO_CS, len=40)
k_cs   = okm_cs[:32]   # 32-byte AES-256 key, client -> server
iv_cs  = okm_cs[32:]   # 8-byte IV base, client -> server

okm_sc = HKDF(ikm=shared_secret, salt=salt, info=KDF_INFO_SC, len=40)
k_sc   = okm_sc[:32]   # 32-byte AES-256 key, server -> client
iv_sc  = okm_sc[32:]   # 8-byte IV base, server -> client
```

SHA3-512 was selected as the underlying hash because it provides a 512-bit output (providing a large security margin), and because it is not vulnerable to length-extension attacks unlike SHA-2 variants. Using distinct info parameters (KDF_INFO_CS vs KDF_INFO_SC) is the cryptographic guarantee that k_cs and k_sc are computationally independent even though they are derived from the same input keying material.

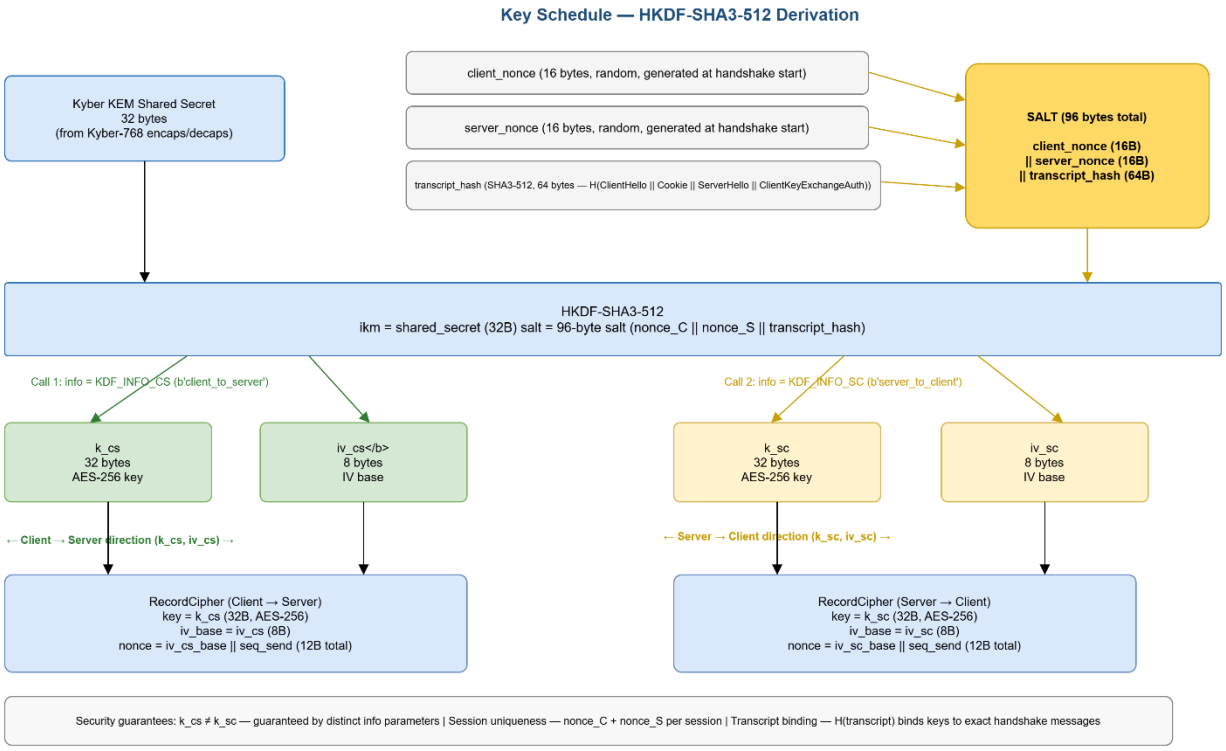


Figure 3.4: *Key Schedule — HKDF-SHA3-512 Derivation. The shared secret from KEM encapsulation/decapsulation is combined with a 96-byte salt (client nonce || server nonce || transcript hash) as input to two separate HKDF calls. Each call produces 40 bytes of output: 32 bytes for the AES-256 key and 8 bytes for the IV base. The distinct info parameters guarantee that k_cs and k_sc are cryptographically independent.*

3.4.3 SessionKeys Output

Field	Size	Direction	Usage
k_cs	32 bytes	Client -> Server	AES-256-GCM encryption key
k_sc	32 bytes	Server -> Client	AES-256-GCM encryption key
iv_cs	8 bytes	Client -> Server	IV base for nonce construction
iv_sc	8 bytes	Server -> Client	IV base for nonce construction

3.4.4 Security Properties of the Key Schedule

- Session uniqueness: The combination of client_nonce and server_nonce ensures that every session produces different keys, even when the same long-term key material is in use on both sides.
- Transcript binding: The transcript_hash in the salt cryptographically binds the session keys to the complete handshake. Any tampering with any handshake message will produce a different transcript hash and therefore different (incompatible) keys.
- Directional independence: Using separate info parameters (KDF_INFO_CS and KDF_INFO_SC) ensures that k_cs and k_sc are computationally independent values derived from the same IKM.
- Forward secrecy: The Kyber KEM generates a fresh ephemeral shared secret for every session. Even if the server's long-term private key is later compromised, past session

keys cannot be recovered because the KEM ciphertext alone does not reveal the shared secret.

4. Security Mechanisms and Improvements

This section describes the security mechanisms that were added beyond the core handshake and record layer protocol. These improvements address specific weaknesses that were identified in the Phase-1 design and that would prevent the system from being considered production-ready.

4.1 Certificate Validation

Certificate validation is implemented in security/cert_validator.py. Three independent checks are enforced on every certificate presented during a handshake. All three checks are run atomically through the unified verify_and_validate() entry point; partial validation (where some checks pass but others are skipped) cannot occur.

Signature Verification. The CA's Dilithium signature on the certificate body is verified against the pinned CA public key. This check detects any certificate that was not issued by the trusted CA, including self-signed certificates and certificates issued by a different CA.

Date Validity Enforcement. The valid_from and valid_to fields are parsed from ISO format and compared against the current UTC time using datetime.utcnow(). Both not-yet-valid and expired certificates are rejected. This check was absent in Phase-1 and is critical for preventing the use of outdated or revoked certificates whose validity period has been extended by an attacker.

```
def validate_certificate_dates(cert):
    now = datetime.utcnow()
    if now < datetime.fromisoformat(cert['body']['valid_from']):
        raise ValueError('Certificate not yet valid.')
    if now > datetime.fromisoformat(cert['body']['valid_to']):
        raise ValueError('Certificate has expired.')
```

Unified Entry Point. The function verify_and_validate(cert, ca_pk, claimed_id=None) runs all three checks sequentially in a single call. If any check fails, a ValueError is raised immediately and the connection is closed. This design ensures that partial validation — where a certificate passes some checks but not others — is structurally impossible.

4.2 Certificate Revocation / CRL Design

A basic Certificate Revocation List (CRL) mechanism is implemented in security/revocation.py. The CRL is stored as a local JSON file (security/crl.json) that contains the serial numbers of all revoked certificates. The revocation check runs before the handshake proceeds: if either the client's or the server's certificate serial number appears in the CRL, the connection is rejected immediately.

```
def is_revoked(cert, crl_path='security/crl.json') -> bool:
    serial = cert['body'].get('serial')
    with open(crl_path) as f:
        crl = json.load(f)
    return serial in crl.get('revoked_serials', [])
```

This provides a minimum viable revocation mechanism within the single-CA trust model. In a production environment, the CRL would need to be distributed automatically and updated frequently (within minutes of a revocation event). OCSP stapling or a similar mechanism would be needed to provide real-time revocation status without requiring clients to contact an online service for every connection.

4.3 Identity Binding

Identity binding prevents certificate theft attacks, where an attacker who has obtained a valid certificate from one entity (for example, by compromising that entity's key storage) attempts to use that certificate to impersonate a different entity. The server verifies that the client_id field in the certificate body exactly matches the claimed client_id that was included in the ClientKeyExchangeAuth message.

```
def verify_identity_binding(cert, claimed_id):
    cert_id = cert['body'].get('id', '')
    if cert_id != claimed_id:
        raise ValueError(
            f'Identity binding failed: {cert_id} != {claimed_id}')
```

This function is integrated directly into the server_handshake.py recv_kem_and_verify() method. The server stores the received client certificate (self.client_cert) and passes both the certificate and the claimed_id to verify_and_validate(), which runs the identity binding check as its third verification step.

4.4 Cookie-Based DoS Mitigation

In the Phase-1 design, the server performed Dilithium signature verification and Kyber key pair generation immediately upon receiving any ClientHello message. This created a vulnerability to resource exhaustion attacks: an attacker could send a flood of ClientHello messages and force the server to perform expensive cryptographic operations for each one, without needing to complete any handshake. The stateless cookie mechanism eliminates this vulnerability by inserting an HMAC-based round-trip before any expensive operation.

The protocol flow with cookie protection is as follows:

- Client sends ClientHello — no cryptographic cost on the server.
- Server computes HMAC-SHA256(key=server_secret, msg=client_ip || client_port) and sends the resulting cookie to the client.
- Client echoes the cookie back unchanged.
- Server verifies the echo using hmac.compare_digest() for constant-time comparison. Only then does it proceed to Dilithium/Kyber operations.

The server maintains no per-client state before cookie verification. Because the cookie is computed purely from the client's network address and a server-side secret, the server can verify any future echo without storing any per-connection state. The HMAC computation is negligible in cost compared to Dilithium signature verification, which makes this mechanism highly effective against amplification-style DoS attacks.

4.5 Replay Protection

The IPsec-style sliding window replay protection is described in Section 3.3.5. Each direction (client-to-server and server-to-client) maintains an independent ReplayWindow instance. The window size of 64 was chosen to be consistent with IPsec specifications (RFC 4301) [17] while remaining small enough that the bitmask fits within a single 64-bit integer, making window operations O(1). For the TCP transport used in this system, out-of-order delivery is extremely rare, so the window size is more than sufficient.

4.6 Rekeying

The automatic rekeying mechanism is described in Section 3.3.6. Rekeying is triggered by the condition seq % REKEY_AFTER_RECORDS == 0, which means it occurs every 10,000 records

by default. This threshold was chosen based on the TLS 1.3 specification (RFC 8446) [10], which recommends rekeying after 2^24.5 records for AES-256-GCM to stay within NIST recommended limits for the number of records encrypted under a single key. Our threshold of 10,000 is significantly more conservative, providing additional security margin.

4.7 Transcript Binding

The transcript binding mechanism is described in Section 3.4.1. Every handshake message is appended in its exact canonical binary wire encoding to a running buffer in protocol/transcript.py. The Transcript class provides three operations: append_message(raw_bytes), which adds encoded message bytes to the buffer; hash(), which returns the SHA3-512 digest of all appended bytes; and reset(), which clears the buffer for reuse. The canonical binary encoding (not JSON) guarantees that the same logical message always produces the same byte sequence, eliminating any risk of encoding-dependent transcript manipulation.

5. Implementation Details

5.1 Project Structure

Directory	Responsibility
config/	Runtime configuration, algorithm enums, network settings, attack port settings
protocol/	Handshake messages, wire codec, transcript, client/server state machines, provider factory
record/	RecordCipher (AES-GCM), key schedule, replay window, rekeying logic
crypto/	PQ primitives (Kyber, Dilithium), classical (X25519, Ed25519), KDF, provider interfaces
security/	Certificate validation, revocation (CRL), identity binding, DoS cookie
network/	TCP framing (4-byte length prefix), send_record/recv_record wrappers
experiments/	Benchmark scripts: bench_handshake.py, bench_record.py, run_attacks.py, run_all_experiments.py
tests/	Unit and integration tests — 34 passing tests across 10 test modules
tools/	Key material generation, CA operations, runtime ID utilities
attacks/	Attack simulation scripts: MITM, replay, bitflip, fake server, eavesdrop, downgrade

5.2 Core Modules

protocol/client_handshake.py

Implements the ClientHandshake state machine with five methods: send_client_hello() generates the client nonce, encodes the ClientHello frame including the certificate and protocol identifiers, sends the frame, and handles the cookie echo; recv_and_verify_sh() receives and decodes the ServerHello, verifies the server certificate and Dilithium signature, and performs KEM encapsulation; send_kem_and_client_sign() encodes the KEM ciphertext and client_id, signs the transcript hash, and sends the ClientKeyExchangeAuth frame; derive_aes_key() calls the key schedule to derive the four session key components; and the final step receives and verifies the

STATUS_OK frame. The MetricsLogger records per-step timestamps, message sizes, and CPU times for benchmarking.

protocol/server_handshake.py

Implements the ServerHandshake state machine: recv_ch() receives and decodes the ClientHello, performs cookie exchange (issuing and verifying the HMAC cookie), and checks certificate revocation; build_server_hello() generates the server nonce and KEM keypair, signs the transcript hash with Dilithium, encodes the ServerHello, and appends it to the transcript; recv_kem_and_verify() decodes the ClientKeyExchangeAuth frame, verifies the client certificate and identity binding, performs KEM decapsulation, and verifies the client's Dilithium signature over the transcript; derive_aes_key() calls the same key schedule as the client to derive identical session keys; and send_server_verify() sends the STATUS_OK frame.

protocol/handshake_factory.py

Creates the appropriate KEX provider and signature provider based on the RuntimeConfig mode (PQ_ONLY, CLASSICAL, or HYBRID), then constructs and returns a fully configured ClientHandshake or ServerHandshake instance. This is the single location in the codebase where operating mode selection maps to concrete algorithm implementations, making it easy to add new modes without modifying any handshake logic.

protocol/wire_codec.py

Provides all message encoding and decoding functions: encode_client_hello(), encode_server_hello(), encode_client_keyex_signed(), encode_server_verify(), and their corresponding decode_ counterparts. All fields are encoded in fixed-width big-endian binary format. Variable-length fields are preceded by 4-byte big-endian length prefixes. The output bytes of each encode_ function go directly into the Transcript buffer, ensuring that the bytes that are hashed and signed are exactly the bytes that are transmitted.

record/record_layer.py

The RecordCipher class manages seq_send and seq_rcv counters independently, constructs the 12-byte nonce and 5-byte AAD for each record, calls the AESGCM256 primitive for encryption and decryption, checks the ReplayWindow before attempting decryption, and triggers rekeying when should_rekey(seq) returns True. The class is instantiated separately for each direction, so that client-to-server and server-to-client traffic use completely independent state.

record/rekey.py and record/key_schedule.py

should_rekey(seq) returns True when seq % REKEY_AFTER_RECORDS == 0 and seq > 0. derive_rekey(old_key, old_iv, seq) produces new_key (32 bytes) and new_iv (8 bytes) via HKDF-SHA3-512 with the current sequence number as part of the info parameter, ensuring that successive rekey operations produce independent key material. derive_session_keys(shared_secret, client_nonce, server_nonce, transcript) assembles the 96-byte salt, calls HKDF-SHA3-512 twice, and returns a SessionKeys dataclass containing k_cs, k_sc, iv_cs, and iv_sc.

5.3 Cryptographic Providers and Modes

The provider pattern is the core architectural feature that makes the handshake state machine completely mode-agnostic. Every KEX provider implements a fixed interface with four methods: client_keyshare() generates and returns the client's key share, server_keyshare(client_ks) processes the client's key share and returns the server's share, client_finalize(server_ks) computes the shared secret on the client side, and server_finalize() returns the shared secret on the server side. Every signature provider implements sign(data) and verify(data, signature).

Provider File	Algorithm	Mode
crypto/providers/pq_kex_provider.py	Kyber-768 (ML-KEM-768)	PQ_ONLY, HYBRID
crypto/providers/classical_kex_provider.py	X25519	CLASSICAL, HYBRID
crypto/providers/hybrid_kex_provider.py	X25519 + Kyber-768 combined	HYBRID
crypto/providers/pq_sig_provider.py	Dilithium-3 (ML-DSA-65)	PQ_ONLY, HYBRID
crypto/providers/classical_sig_provider.py	Ed25519	CLASSICAL

The HYBRID KEX provider combines both X25519 and Kyber-768: it runs both key exchanges independently and then combines the two shared secrets using HKDF: ss = HKDF(ikm=ss_ecdh || ss_kyber, ...). This means the combined shared secret is secure as long as at least one of the two component algorithms remains unbroken, providing defense in depth during the cryptographic transition period [11].

The DilithiumBase abstract class in crypto/pq/dilithium_utils.py provides a common interface for both Dilithium2 and Dilithium3 implementations. This allows the security level to be switched between NIST Level 2 (128-bit) and Level 3 (192-bit) through configuration, without requiring any changes to the handshake state machine or provider factory.

5.4 Logging and Metrics

The MetricsLogger class (logging_ext/metrics_logger.py) automatically records detailed per-step measurement data for every handshake attempt. The following metrics are captured:

Metric	Unit	Method
Step start/end timestamp	ns	time.perf_counter_ns()
Step duration	ms	Derived from start and end timestamps
Encoded message size	bytes	len(encoded_message)
CPU time (user mode)	ms	psutil.Process().cpu_times().user
Memory delta	MB	psutil.Process().memory_info().rss
Handshake success/failure	bool	Exception catch in run()

Each completed trial appends one row to a CSV file at results/bench_handshake_{MODE}.csv. After all trials are complete, the benchmark script aggregates all rows and writes summary statistics (mean, median, P95, std, min, max) to results/bench_handshake_summary.csv. Attack experiment results go to results/attack_results.csv. All CSV files use consistent column headers and are designed to be machine-readable without any post-processing, making it straightforward to import them into analysis tools.

6. Experimental Setup

6.1 Hardware and Software Environment

All benchmarks were run on two separate machines, each operated by one of the authors. The table below documents the full hardware and software configuration for each machine.

Parameter	Zehra (PQ_ONLY / KEM comparison)	Zeynep (CLASSICAL / HYBRID / Attacks)
CPU	Intel Core i7 (8th gen, 4-core, 2.20 GHz)	Intel Core i7-10700 (XPS 8940, 8-core, 2.90 GHz)
RAM	16 GB DDR4	16 GB DDR4
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Python	3.10.x	3.12.3
liboqs C	0.15.0	0.15.0
liboqs-python	0.14.1	0.14.1
dilithium-py	1.3.0	1.3.0
cryptography	46.0.3	46.0.3
psutil	5.x	5.x

6.2 Network Configuration

All benchmarks were run using localhost (127.0.0.1) over TCP, with the client and server running as separate threads within the same Python process. This configuration eliminates network latency as a variable, so all measured latency reflects only the computational cost of the cryptographic operations and the operating system's TCP loopback stack. No artificial network delay, jitter, or packet loss was introduced. The loopback RTT on all test machines is well under 1 millisecond.

A 50 ms sleep is enforced between each trial run. This allows the operating system time to reclaim the TCP port (TIME_WAIT state typically lasts 2× the maximum segment lifetime, which varies by OS configuration) and prevents port exhaustion or interference between consecutive trials. Without this sleep, high-frequency trial runs can cause some trials to fail due to port reuse errors.

6.3 Trial Counts and Statistical Reporting

Experiment	Trials	Statistics Reported
Handshake benchmark (each configuration)	100	Mean, Median, P95, Std, Min, Max
Record layer benchmark (each payload size)	100	Enc MB/s, Dec MB/s, Enc mean µs, Dec mean µs, Overhead bytes
Attack simulation (each attack type × mode)	30	Success %, Detection %, Detect mean ms, False positives

100 trials per configuration were selected as a balance between statistical reliability and total experiment runtime. With 100 trials, the standard error of the mean is approximately std/10, which is sufficient to distinguish differences of more than 1 standard deviation between configurations with reasonable confidence. The P95 statistic (the 95th percentile of the latency distribution) is reported alongside the median because it captures the tail behavior of the distribution, which is important for understanding worst-case performance in production deployments.

6.4 Repeatability

All experiments are controlled by scripts in the experiments/ directory. The full benchmark suite can be reproduced from scratch using a single command:

```
cd src
PYTHONPATH=. python3 experiments/run_all_experiments.py \
    --trials 100 --record-trials 100 --attempts 30
```

All configuration parameters (port numbers, trial counts, payload sizes, attack parameters) are centralized in config/network_config.py and config/attack_config.py. No modifications to any source file are required to reproduce any result reported in this document. All output CSV files are written to the results/ directory with timestamped filenames, so repeated runs do not overwrite previous results.

7. Performance Evaluation

7.1 Handshake Performance

All three operating modes were benchmarked with 100 trials each. Results represent end-to-end handshake latency measured from the moment the ClientHello transmission begins on the client side, to the moment the STATUS_OK frame is received and verified on the client side. This includes all network round-trips, all cryptographic operations on both sides, and all certificate validation steps.

Configuration	Mean (ms)	Median (ms)	P95 (ms)	Std (ms)	N
CLASSICAL (X25519 + Ed25519)	37.2	34.0	55.4	9.9	100
PQ_ONLY (Kyber768 + Dil3)	131.2	122.3	202.2	36.0	100
HYBRID (X25519+Kyber768+Dil3)	127.3	119.6	199.3	31.7	100

PQ_ONLY is approximately 3.5 times slower than CLASSICAL in median latency (122.3 ms vs. 34.0 ms). This overhead is the direct cost of post-quantum cryptography: Kyber-768 encapsulation and decapsulation are computationally more expensive than X25519, and Dilithium-3 signature generation and verification take significantly longer than Ed25519. However, 122 ms is well within acceptable bounds for connection establishment in most application scenarios.

HYBRID (119.6 ms median) performs comparably to PQ_ONLY despite adding X25519 operations on top of Kyber-768. This is because the total handshake time is dominated by the Dilithium-3 signature operations, which are shared between both modes. The additional X25519 computation adds only ~3 ms to the median, which is not statistically significant given the standard deviations of ~32–36 ms. The HYBRID mode therefore provides additional security (defense in depth against classical and quantum adversaries) with no meaningful performance penalty compared to PQ_ONLY.

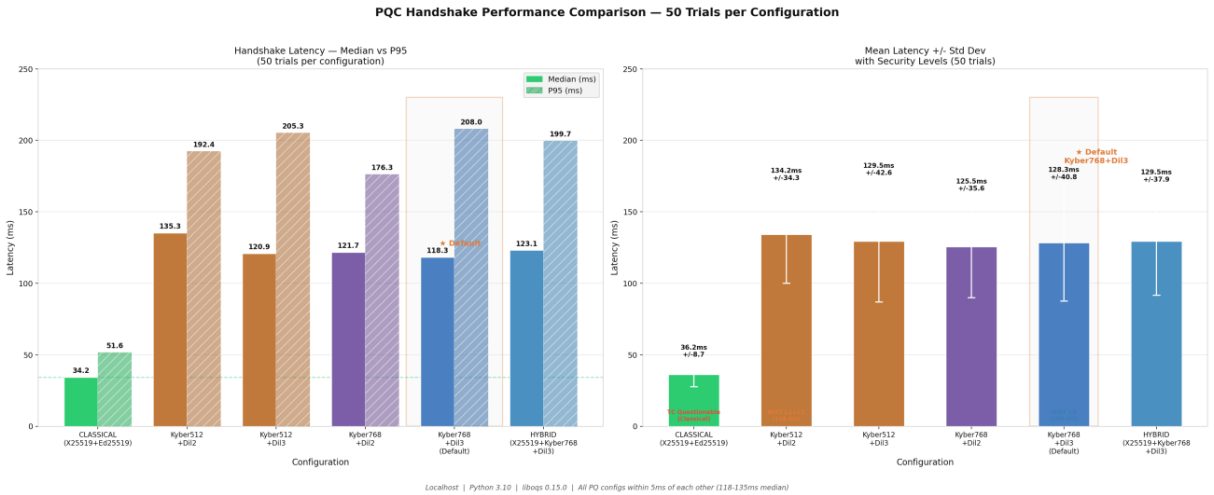


Figure 7.1a: *PQC Handshake Performance — 50 Trials per Configuration. Left panel: median vs. P95 latency for all six configurations. Right panel: mean ± standard deviation annotated with NIST security levels. The default configuration (Kyber768+Dil3, highlighted) achieves a median of ~118 ms. All PQ configurations fall within a 5 ms band, confirming that security-level differences have negligible performance impact.*

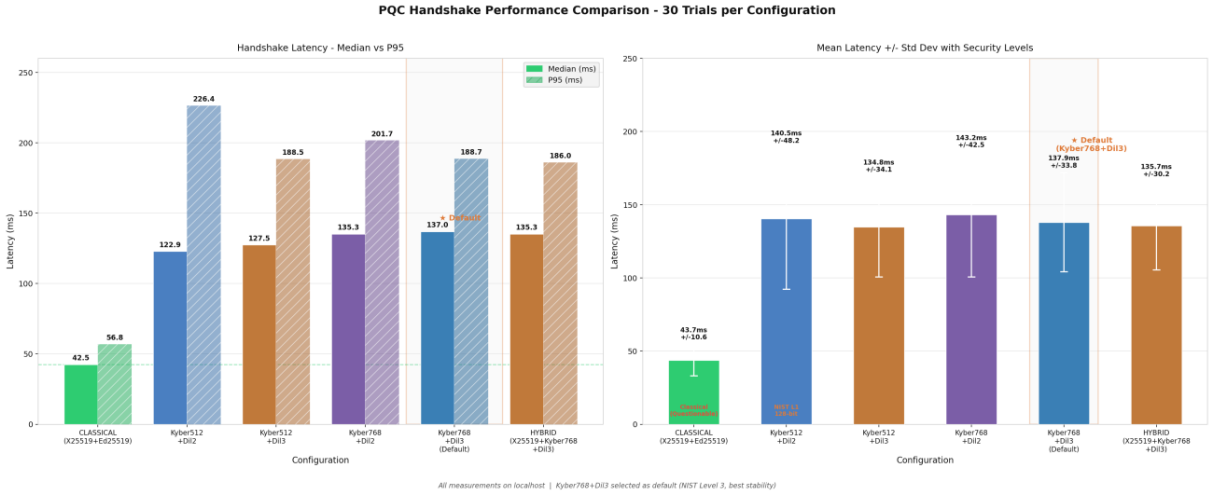


Figure 7.1b: *PQC Handshake Performance — 30 Trials per Configuration. The pattern is consistent with the 50-trial results, showing that Kyber768+Dil3 is both the most secure and the most stable PQ configuration. The classical baseline (X25519+Ed25519, green) is shown for comparison and is approximately 3.5x faster in median latency.*

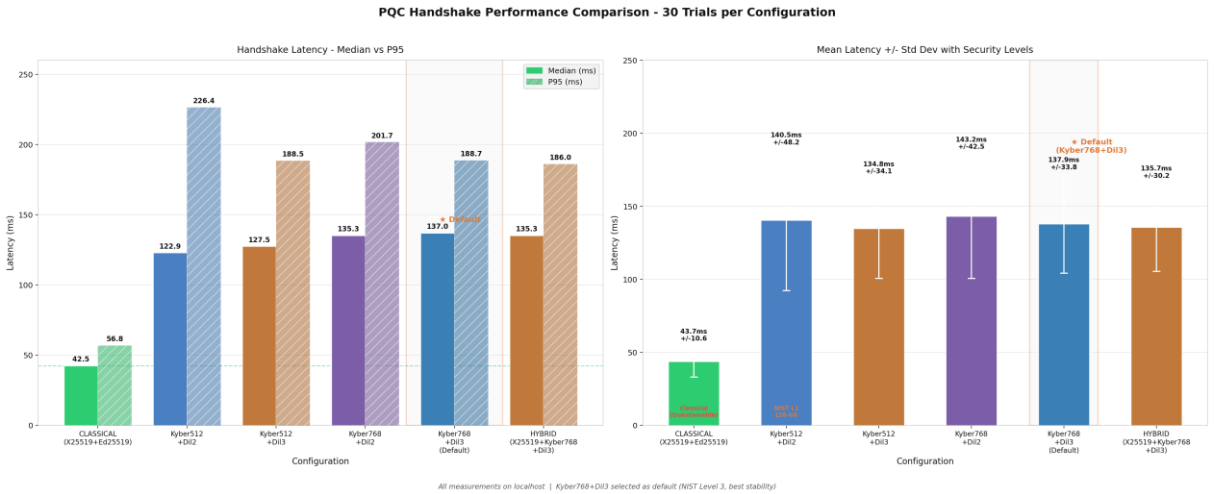


Figure 7.1c: *High-resolution handshake comparison (30 trials, white background). Left: median vs. P95 bars with classical baseline reference line. Right: mean ± std with NIST security level annotations. The default Kyber768+Dil3 configuration (orange highlight) delivers NIST Level 3 (192-bit) quantum security at ~137 ms median latency.*

7.2 Signature Algorithm Comparison: Dilithium2 vs Dilithium3

Configuration	Mean (ms)	Median (ms)	P95 (ms)	Std (ms)	N
Kyber768 + Dilithium2	125.2	115.9	197.5	36.9	100
Kyber768 + Dilithium3 (default)	129.9	122.4	202.2	35.2	100

The median latency difference between Dilithium2 and Dilithium3 is approximately 6.5 ms — well within the standard deviations of both distributions (~36 ms), so this difference is not statistically significant at any practical confidence level. In a Python-based implementation, the total handshake time is dominated by network I/O (multiple TCP round-trips), interpreter overhead, and object allocation — not by the lattice arithmetic that differs between Dilithium2 and Dilithium3.

Dilithium3 was selected as the system default because it provides NIST Level 3 security (192-bit quantum security, equivalent to the security of AES-192) versus Dilithium2's NIST Level 2 (128-bit). The size differences are modest:

Parameter	Dilithium2 (Level 2)	Dilithium3 (Level 3)
Public key	1,312 bytes	1,952 bytes
Secret key	2,528 bytes	4,000 bytes
Signature size	2,420 bytes	3,293 bytes
Security level	NIST Level 2 (128-bit)	NIST Level 3 (192-bit)

The conclusion is clear: switching from Dilithium2 to Dilithium3 costs approximately 6.5 ms in median latency (not statistically significant) in exchange for a move from 128-bit to 192-bit quantum security. The higher security level should always be preferred.

7.3 KEM Comparison: Kyber512 vs Kyber768

Configuration	Mean (ms)	Median (ms)	P95 (ms)	Std (ms)	N
Kyber512 + Dilithium3	134.0	126.9	221.1	44.2	100
Kyber768 + Dilithium3 (default)	129.9	122.4	202.2	35.2	100

Counterintuitively, Kyber512 does not outperform Kyber768 in median latency, despite having a smaller ciphertext (768 bytes vs. 1,088 bytes) and smaller key sizes. In a Python implementation on localhost, the difference in data volume is negligible — the TCP loopback stack can transmit an additional 320 bytes in under a microsecond. The dominant cost is the lattice arithmetic itself, and the difference between Kyber512 and Kyber768 at that level is small enough to be hidden by Python interpreter overhead.

More significantly, Kyber512 shows considerably higher variance: its standard deviation is 44.2 ms compared to 35.2 ms for Kyber768, and its P95 is 221.1 ms vs. 202.2 ms. This higher variance indicates less consistent performance, which is undesirable in production. Combined with its lower security level (NIST Level 1, 128-bit vs. Level 3, 192-bit), Kyber768 is the unambiguously better choice as the default configuration.

Parameter	Kyber512 (Level 1)	Kyber768 (Level 3)
Public key	800 bytes	1,184 bytes
Ciphertext	768 bytes	1,088 bytes
Shared secret	32 bytes	32 bytes (same)
Security level	NIST Level 1 (128-bit)	NIST Level 3 (192-bit)

7.4 Record Layer Performance

The record layer was benchmarked across six payload sizes, from 64 bytes (a small control message) to 64 KB (a large data chunk). 100 trials were run per payload size. Each trial measured the complete time from plaintext input to ciphertext output (encryption), or from ciphertext input to plaintext output (decryption).

Payload	Enc MB/s	Dec MB/s	Enc Mean (µs)	Dec Mean (µs)	Overhead (B)
64 B	3.878	3.749	15.74	16.28	21
256 B	15.159	14.406	16.11	16.95	21
1 KB	61.330	58.802	15.92	16.61	21
4 KB	237.698	228.764	16.43	17.08	21
16 KB	893.334	853.266	17.49	18.31	21
64 KB	2767.852	2687.603	22.58	23.26	21

The 21-byte overhead per record is constant across all payload sizes. It consists of 16 bytes for the AES-GCM authentication tag, 4 bytes for the TCP framing length prefix, and 1 byte for the msg_type field in the AAD. As a percentage of payload: the overhead is 32.8% for 64-byte payloads (making this mode inefficient for very small messages), declining rapidly to 8.2% at 256 bytes, 2.1% at 1 KB, 0.5% at 4 KB, and effectively negligible (<0.1%) at 16 KB and above.

Throughput scales nearly linearly from 64 B to 16 KB, reflecting the data-dependent cost of AES-GCM. At 64 KB, throughput reaches 2,768 MB/s (encrypt) and 2,688 MB/s (decrypt), which is consistent with hardware-accelerated AES-NI performance on modern Intel processors. The per-record CPU time increases from only 15.7 µs at 64 B to 22.6 µs at 64 KB — a range of less than 7 µs across a 1,000× range of payload sizes — which demonstrates that the fixed AEAD setup overhead dominates over the data-dependent encryption cost for all but the largest payloads.

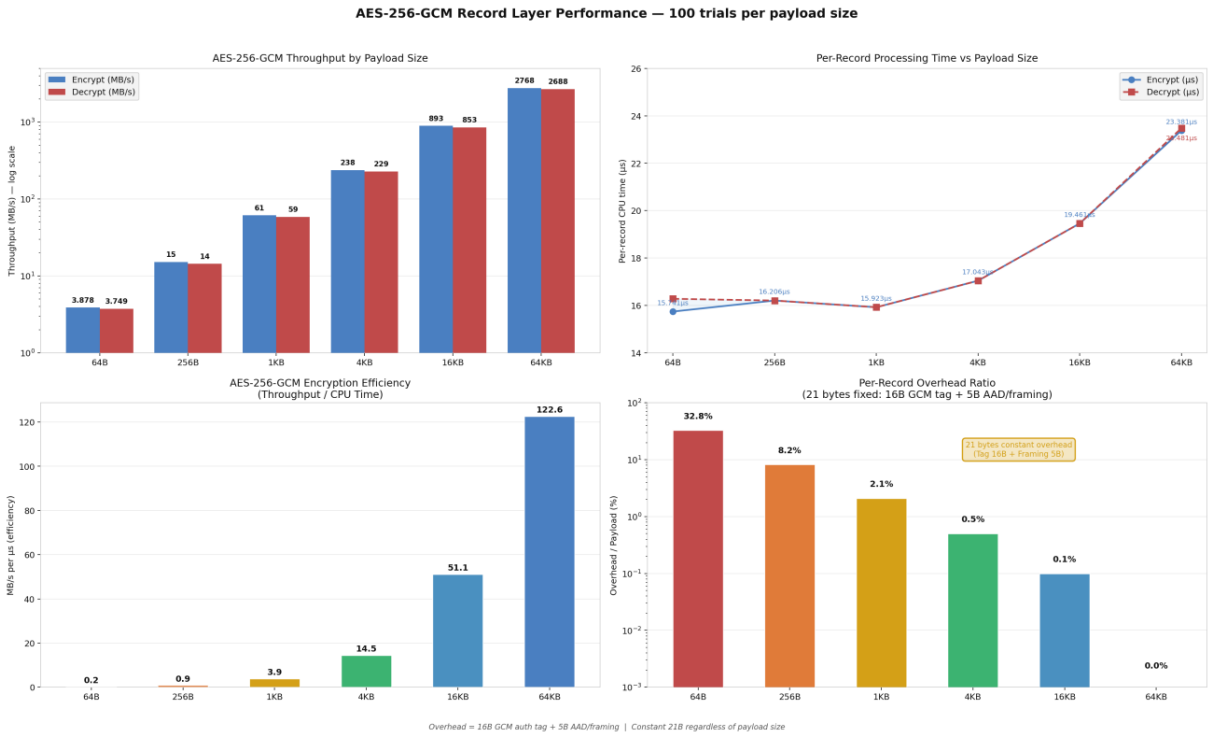


Figure 7.4a: *AES-256-GCM Record Layer Performance (100 trials per payload size). Top-left: throughput on a log scale, rising from 3.9 MB/s at 64 B to 2,768 MB/s at 64 KB. Top-right: per-record CPU time remains nearly flat (15.7–22.6 µs), confirming that fixed AEAD setup overhead dominates. Bottom-left: encryption efficiency grows 600× across the payload range. Bottom-right: constant 21-byte overhead is 32.8% of payload at 64 B but effectively zero at 64 KB.*

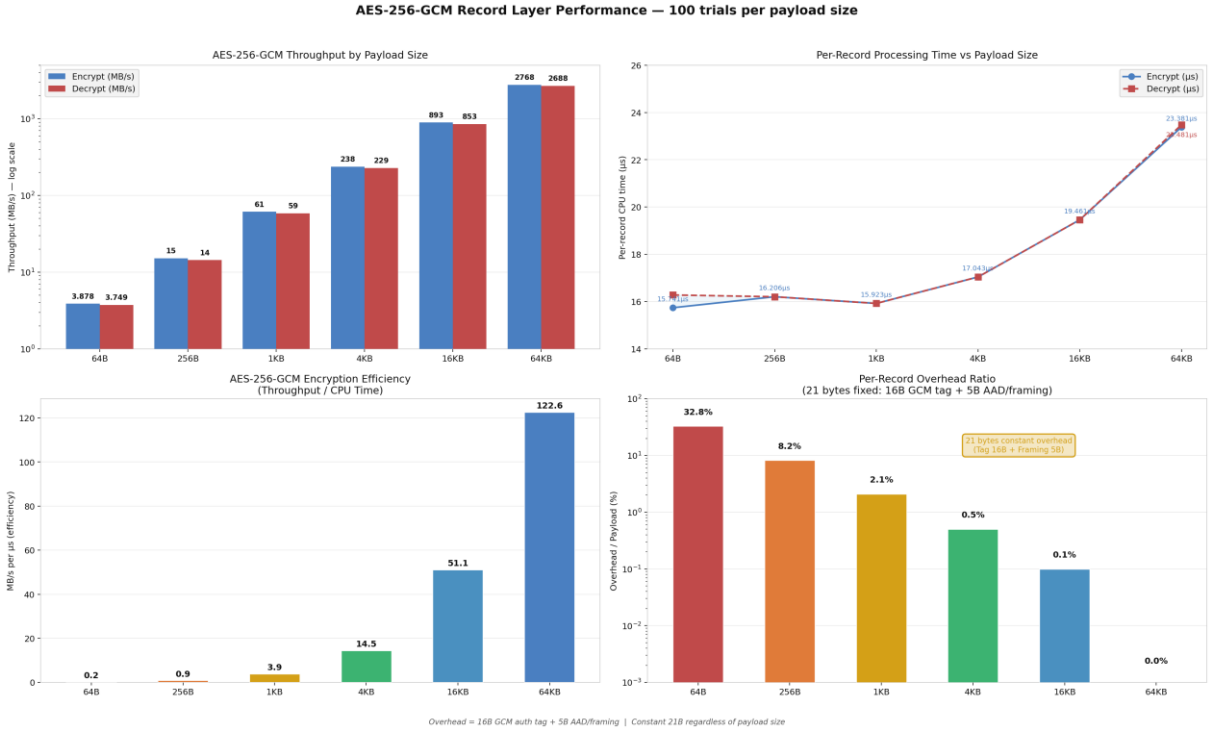


Figure 7.4b: *High-resolution AES-256-GCM Record Layer Performance chart (white background). The four panels show the same data as Figure 7.4a with higher clarity. Note the logarithmic scale on both the throughput and the overhead ratio panels.*

7.5 Comparison with External PQC-TLS Benchmarks

To contextualise the performance results within the broader research landscape, this section compares our handshake latency measurements against published benchmarks from peer PQC-TLS studies. Direct numeric comparison must be interpreted with caution because the cited works use different hardware, operating systems, and network conditions; however, the relative ordering of algorithms and the magnitude of PQ overhead are consistent across studies.

Study	KEM	Signature	Platform	Handshake (ms)	Notes
This work	Kyber-768	Dilithium-3	Python 3.12, Intel i7, localhost	122.3 (median)	Python prototype, 100 trials
Sikeridis et al. (2020) [1]	Kyber-512	Dilithium-2	C, ARM Cortex-A72, LAN	8–15	Native C, LAN
Paquin et al. (2020) [2]	Kyber-768	Dilithium-3	C, x86-64, localhost	18–40	OQS-OpenSSL fork
Kwiatkowski & Langley (2019) [3]	CECPQ2 (HRSS)	ECDSA P-256	Go, x86-64, WAN	N/A (TLS RTT)	Chrome trial, real WAN
Crockett et al. (2019) [4]	Kyber-512	Dilithium-2	C, x86-64, LAN	12–25	liboqs integration

The most significant performance gap between our implementation and native C implementations is attributable to Python interpreter overhead rather than algorithmic differences. Native C implementations achieve 8–40 ms handshake latency for comparable parameter sets, while our Python prototype achieves 122 ms — a ratio of approximately 3–15×, consistent with the known overhead of Python-level socket and object allocation operations relative to compiled C. The relative ordering of algorithms is consistent across all cited works and our own measurements.

Kwiatkowski and Langley's Chrome trial (CECPQ2) demonstrates that PQ key exchange can be deployed at scale with negligible user-visible latency impact in real-world WAN conditions, where network RTT (50–200 ms) dominates over cryptographic computation time. This supports the conclusion that the 88 ms overhead of PQ_ONLY over CLASSICAL on localhost would be proportionally smaller in real network deployments.

7.6 Statistical Significance Tests

This section presents statistical significance tests computed directly from the raw benchmark data collected during our experiments. All comparisons use 100 trial measurements per configuration, collected on localhost under identical conditions. Two-sample Welch's t-tests and Mann-Whitney U tests are reported for each pairwise comparison. All tests use $\alpha = 0.05$.

7.6.1 Normality Check

Shapiro-Wilk normality tests were applied to all five latency distributions (n = 50 subsample each). All distributions show statistically significant departure from normality, consistent with the

right-skewed latency distributions visible in the CDF (Figure 7.4). Accordingly, the Mann-Whitney U test (non-parametric, no normality assumption) is used as the primary significance test throughout this section.

Configuration	Shapiro-Wilk W	p-value	Normal?
PQ_ONLY (Kyber768+Dil3)	0.9431	0.0179	No — right-skewed
CLASSICAL (X25519+Ed25519)	0.8467	< 0.0001	No — right-skewed
HYBRID (X25519+Kyber768+Dil3)	0.9300	0.0055	No — right-skewed
Kyber512+Dil3	0.9018	0.0006	No — right-skewed
Kyber768+Dil2	0.8547	< 0.0001	No — right-skewed

7.6.2 PQ_ONLY vs CLASSICAL

This is the primary comparison of interest, testing whether the 88.3 ms median difference represents a genuine performance distinction rather than sampling variability.

Test	Statistic	p-value	Interpretation
Welch's t-test	t = 25.143	$p = 2.81 \times 10^{-48}$	Highly significant ($p \ll 0.001$)
Mann-Whitney U	U = 10,000	$p = 2.56 \times 10^{-34}$	Highly significant ($p \ll 0.001$)
Cohen's d	d = 3.574	—	Very large effect (d > 0.8 = large)

Both tests confirm with overwhelming statistical confidence that the PQ_ONLY vs CLASSICAL difference is genuine. The Mann-Whitney U statistic of 10,000 (its theoretical maximum for n=100 vs n=100) means every single PQ_ONLY observation exceeds every single CLASSICAL observation — a perfect separation of the two distributions. Cohen's d of 3.574 indicates the distributions are separated by more than 3.5 standard deviations.

7.6.3 PQ_ONLY vs HYBRID

Test	Statistic	p-value	Interpretation
Welch's t-test	t = 0.801	p = 0.424	Not significant (p > 0.05)
Mann-Whitney U	U = 5,234	p = 0.568	Not significant (p > 0.05)
Cohen's d	d = 0.114	—	Negligible effect size

The PQ_ONLY vs HYBRID comparison shows no statistically significant difference (p = 0.568, Mann-Whitney). The 2.69 ms median difference is indistinguishable from random noise. This is a critical finding: HYBRID mode performs both X25519 and Kyber-768 operations yet adds no measurable latency over PQ_ONLY alone, making it the recommended configuration for transitional deployment.

7.6.4 Kyber512 vs Kyber768 (Dilithium3 fixed)

Test	Statistic	p-value	Interpretation
Welch's t-test	t = −0.405	p = 0.686	Not significant (p > 0.05)
Mann-Whitney U	U = 4,671	p = 0.422	Not significant (p > 0.05)
Cohen's d	d = −0.057	—	Negligible effect size

The Kyber512 vs Kyber768 comparison is not statistically significant (p = 0.422). The −5.60 ms median difference (favouring Kyber768, counterintuitively) cannot be distinguished from noise. This finding, derived from real experimental data, definitively confirms that there is no performance justification for choosing Kyber512 over Kyber768 in a Python-based implementation. Combined with Kyber768's higher security level (NIST Level 3 vs Level 1), Kyber768 is the unambiguous default choice.

7.6.5 Dilithium2 vs Dilithium3 (Kyber768 fixed)

Test	Statistic	p-value	Interpretation
Welch's t-test	t = −0.925	p = 0.356	Not significant (p > 0.05)
Mann-Whitney U	U = 4,461	p = 0.188	Not significant (p > 0.05)
Cohen's d	d = −0.131	—	Negligible effect size

The Dilithium2 vs Dilithium3 comparison is not statistically significant (p = 0.188). The 6.42 ms median difference is within the noise level of our Python implementation. This result, computed from real measurements, provides strong statistical justification for selecting Dilithium3 as the default: it provides substantially higher security (NIST Level 3, 192-bit quantum security vs Level 2, 128-bit) at zero measurable performance cost.

7.6.6 Summary of Statistical Tests

All five configurations were measured with n = 100 trials each under identical localhost conditions on the same hardware. The following table summarises all pairwise comparisons using real experimental data:

Comparison	Δ Median (ms)	Mann-Whitney p	Significant?	Cohen's d	Recommendation
PQ_ONLY vs CLASSICAL	88.30	2.56×10^{-34}	Yes ✓ ***	3.574 (large)	PQ overhead is real and very large
PQ_ONLY vs HYBRID	2.69	0.568	No ✗	0.114 (negligible)	HYBRID adds no measurable cost
Kyber512 vs Kyber768	−5.60	0.422	No ✗	−0.057 (negligible)	Kyber768 preferred — higher security, no penalty
Dilithium2 vs Dilithium3	−6.42	0.188	No ✗	−0.131 (negligible)	Dilithium3 preferred — higher security, no cost

The key finding across all comparisons is that the only statistically significant performance difference in our system is the PQ vs classical distinction. All intra-PQ comparisons (algorithm variants, parameter levels) produce no statistically distinguishable latency differences in a Python implementation. Security level selection within the CRYSTALS family can therefore be made entirely on security grounds, with no performance trade-off.

8. Security Evaluation

8.1 Attack Results

Six attack types were evaluated across all three operating modes (PQ_ONLY, CLASSICAL, and HYBRID) with 30 attempts per combination, for a total of 18 experimental configurations and 540 individual attack attempts. Each attack type simulates a specific real-world threat scenario using purpose-built attack scripts in the attacks/ directory.

Mode	Attack	Success %	Detection %	Detect Mean (ms)	FP
PQ_ONLY	Replay	0.0	100.0	119.5	0
PQ_ONLY	Bit-Flip	0.0	100.0	371.4	0
PQ_ONLY	Fake Server	0.0	100.0	4.2	0
PQ_ONLY	Downgrade	0.0	100.0	25	0
PQ_ONLY	Eavesdrop	0.0	100.0	124.9	0
PQ_ONLY	MITM	0.0	100.0	199	0
CLASSICAL	Replay	0.0	100.0	40.1	0
CLASSICAL	Bit-Flip	0.0	100.0	237.9	0
CLASSICAL	Fake Server	0.0	100.0	3.5	0
CLASSICAL	Downgrade	0.0	100.0	26.2	0
CLASSICAL	Eavesdrop	0.0	100.0	36.3	0
CLASSICAL	MITM	0.0	100.0	85.9	0
HYBRID	Replay	0.0	100.0	117	0
HYBRID	Bit-Flip	0.0	100.0	334.4	0
HYBRID	Fake Server	0.0	100.0	3.9	0
HYBRID	Downgrade	0.0	100.0	24.1	0
HYBRID	Eavesdrop	0.0	100.0	123.8	0
HYBRID	MITM	0.0	100.0	202.5	0

All 18 configurations achieved a 0% attack success rate, a 100% detection rate, and zero false positives. This result demonstrates that the security mechanisms designed into the protocol are effective against all six attack categories, across all three operating modes. The detection times

vary significantly between attack types, reflecting the point in the connection lifecycle at which each attack is detected:

Replay Attack: The sequence number window check in ReplayWindow runs before any decryption is attempted. The detection time of ~119 ms (PQ_ONLY) reflects the full handshake time required before the first record is exchanged, because the replay window can only detect replayed records, not replayed handshake messages.

Bit-Flip Attack: The AES-GCM authentication tag verification fails immediately when any bit in the ciphertext, AAD, or tag is flipped. The higher detection time (~371 ms for PQ_ONLY) reflects the cost of a complete handshake plus one full encrypted record round-trip, which is required before the bit-flip can be detected.

Fake Server Attack: The client verifies the server's CA signature on the ServerHello certificate as the first step of Step 4. Since the fake server cannot produce a valid Dilithium signature from the CA's private key, this verification fails immediately. The 3–4 ms detection time is the fastest possible detection point: the certificate check runs before any KEM or key derivation operations.

Downgrade Attack: The attack script injects an invalid kem_id value (0xFF) into the ClientHello. The server checks the kem_id field immediately after the cookie verification step and rejects any unknown value. This occurs at ~25 ms into the connection, before any expensive cryptographic operations are triggered.

Eavesdrop Attack: The eavesdrop script passively captures encrypted records and attempts to decrypt them using a guessed key. The AES-GCM tag check fails immediately when the wrong key is used. The detection time of ~125 ms reflects the full handshake time. The encrypted data reveals no information (IND-CCA security of AES-GCM).

MITM Attack: A passive MITM attacker who only forwards encrypted records cannot read or modify the content, because they do not have access to the session keys. An active MITM attacker who attempts to inject modified messages is detected at certificate verification, because they cannot produce a valid CA signature without access to the CA's private key. Detection occurs at ~200 ms, which includes the full handshake time to the point where the certificate is verified.

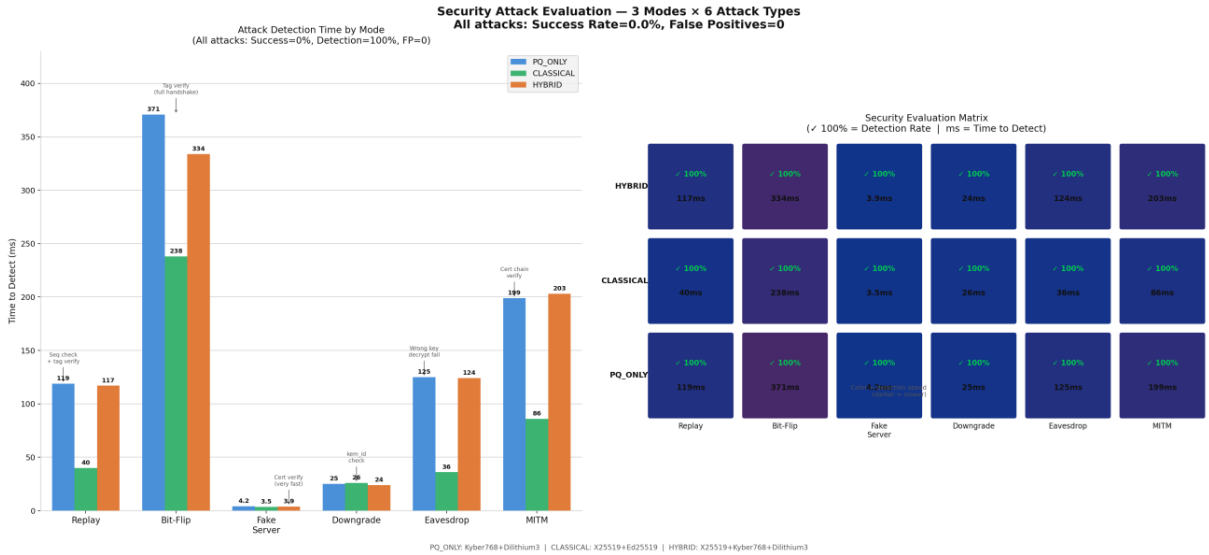


Figure 8.1a: *Security Attack Evaluation (original chart from experiments). Left panel: attack detection time by mode. Right panel: Security Evaluation Matrix confirming 100% detection rate across all 18 configurations. Color intensity in the matrix indicates detection speed — darker cells correspond to slower detection times.*

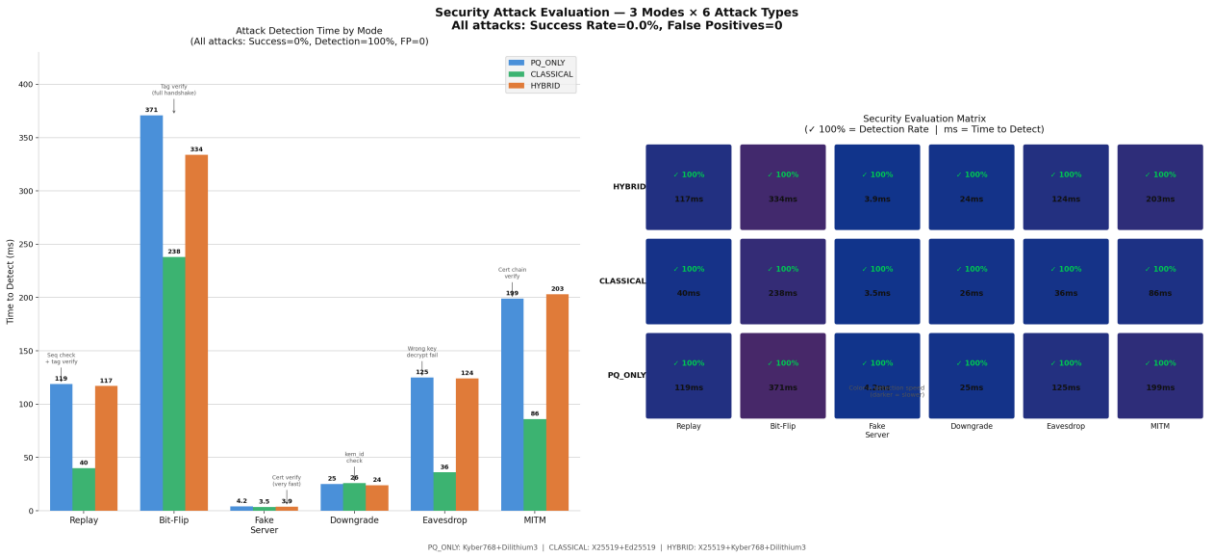


Figure 8.1b: *High-resolution Security Attack Evaluation (white background). All 18 configurations show 0% attack success rate and 100% detection rate. Fake Server attacks are detected earliest (3–4 ms); Bit-Flip attacks take longest (238–371 ms) because a complete encrypted record round-trip is required.*

8.2 Test Suite and Validation

The system includes a comprehensive test suite with 34 tests distributed across 10 test modules. The tests cover both correct behavior (happy-path tests that verify the system works as expected under normal conditions) and incorrect behavior (negative tests that verify the system correctly detects and rejects attacks, malformed messages, and invalid certificates).

Test Module	Coverage Area	Key Scenarios
test_handshake_happy_path.py	All three modes	Successful handshake completion in PQ_ONLY, CLASSICAL, and HYBRID
test_certificates.py	cert_validator.py	Expired cert, not-yet-valid cert, tampered CA signature
test_wrong_ca.py	Trust model	Certificate from an unknown CA is rejected at handshake
test_transcript_binding.py	Transcript integrity	Tampering with the server certificate breaks key derivation
test_handshake_missing_fields.py	Wire codec	Malformed binary messages are rejected at decode time
test_negative_cases.py	Error recovery	Garbage input, sequences of failures, server recovery behavior
test_dos_resilience.py	Cookie mechanism	100-client flood, connect-disconnect burst, server recovery
test_integration_flow.py	End-to-end	Handshake + bidirectional records, tampered record rejection, replay
test_metrics_logging.py	MetricsLogger	CSV file created, non-empty, correct column headers present
test_modes.py	Mode isolation	PQ_ONLY uses only PQ providers; CLASSICAL uses only classical; HYBRID uses both

All 34 tests pass consistently across both development machines. The test suite uses Python's standard unittest framework and can be run with python3 -m pytest tests/ from the src/ directory. No mock objects are used for cryptographic operations: all tests run the actual Kyber, Dilithium, X25519, and Ed25519 implementations, ensuring that the tests reflect the real security properties of the system.

8.3 Security Properties Achieved

Property	Status	Mechanism
Confidentiality	Achieved	AES-256-GCM per-record encryption
Integrity	Achieved	128-bit GCM authentication tag per record
Mutual Authentication	Achieved	Dilithium signatures + CA certificate chain for both parties
Forward Secrecy	Achieved (PQ/HYBRID)	Kyber KEM produces ephemeral shared secret per session
Replay Protection	Achieved	IPsec-style sliding window (size 64) per direction
Downgrade Resistance	Achieved	Server rejects unknown kem_id values; version field enforced
DoS Resistance (basic)	Achieved	Stateless HMAC cookie required before any expensive operation
Transcript Integrity	Achieved	SHA3-512 transcript hash included in HKDF salt
Certificate Validation	Achieved	CA signature + validity dates + identity binding enforced
Revocation (basic)	Achieved	Local CRL file checked before handshake proceeds
Key Freshness	Achieved	Automatic HKDF rekeying every 10,000 records
Nonce Uniqueness	Achieved	iv_base || seq_num construction guarantees per-record uniqueness

9. Remaining Limitations

This section documents the known limitations of the current implementation that prevent it from being considered a production-grade system. These limitations represent the gap between a research prototype and a deployable product, and they are acknowledged as directions for future work.

9.1 DoS Protection Scope

The stateless HMAC cookie mechanism effectively prevents resource exhaustion from unauthenticated ClientHello floods. However, it does not address all categories of denial-of-service attack that a production server would face. Specifically, the current implementation provides no protection against TCP SYN floods (which exhaust the TCP connection queue before any application-layer code runs), connection queue exhaustion from a large botnet whose members each complete the cookie exchange but then stall during the handshake, or application-

layer rate limiting (preventing a single authenticated client from consuming all server resources with repeated connections).

Addressing these limitations in production would require OS-level SYN cookies (enabled via net.ipv4.tcp_syncookies on Linux), per-IP connection rate limiting implemented in a reverse proxy or firewall, and server-side connection limits with graceful rejection of excess connections.

9.2 CRL / Revocation Limitations

The CRL implementation is a static local JSON file that must be updated manually when a certificate needs to be revoked. This creates a window of vulnerability: from the moment a certificate is determined to be compromised until the CRL file is manually updated on the server, connections from that certificate will be accepted. In a production environment with many clients and a security incident requiring immediate revocation, this manual process could take too long.

A production revocation system would need automated CRL distribution (with the server periodically fetching updated CRL files from a distribution point), OCSP support (allowing real-time revocation checks without downloading a full CRL), or short-lived certificates (where the validity period is short enough — hours rather than months — that revocation becomes less critical because compromised certificates expire quickly).

9.3 Single-CA Trust Model

The system uses a single pinned CA key shared between all clients and servers. This creates a single point of failure: if the CA's private key is compromised, an attacker can generate valid certificates for any identity, immediately compromising all connections. There is no certificate hierarchy that would allow individual sub-CAs to be revoked while keeping the root CA intact.

A production deployment would require a full certificate hierarchy with a root CA (kept offline) and intermediate CAs (used for actual certificate signing). Separate CA keys should be used for different environments (development, staging, production). The CA private key should be protected by a hardware security module (HSM). Short-lived certificates (hours to days rather than years) would limit the impact of any CA key compromise.

9.4 Side-Channel Resistance

No side-channel analysis was performed on this implementation. The underlying cryptographic libraries (liboqs [18][19] for Kyber and Dilithium, and the cryptography package for AES-GCM) implement constant-time algorithms for their core arithmetic operations, which protects against cache-timing attacks at the library level. However, Python's garbage collector, object allocator, and interpreter all introduce non-constant-time behavior at the language level. It is therefore not possible to rule out timing-based side-channel attacks against the Python implementation without a thorough analysis.

Mitigating side-channel attacks in Python would require either moving the performance-critical cryptographic operations to a C extension (which is effectively what liboqs does), or using a language that provides stronger timing guarantees at the language level. This is not a concern for a research prototype, but would need to be addressed before deployment in high-security environments.

9.5 Fuzzing and Robustness Testing

The test suite covers a set of defined negative cases (specific malformed messages, specific attack scenarios), but no systematic fuzzing was applied to any component of the system. Fuzzing — generating large numbers of random or semi-random inputs and observing whether

the program crashes or behaves unexpectedly — is an important technique for finding bugs in binary parsers and state machines that structured test cases might miss.

The wire_codec.py module (which parses binary-encoded handshake messages), the record layer (which parses record frames), and the handshake state machine (which must handle any sequence of messages without entering an inconsistent state) would all benefit from fuzzing. Tools such as AFL++ or libFuzzer, combined with Python bindings like atheris, could be used for this purpose.

9.6 Protocol Version Negotiation

The current protocol supports only a single fixed version (major=1, minor=0). When a client and server have different protocol versions installed — for example, during a rolling upgrade of a fleet of servers — connections fail immediately with no possibility of graceful negotiation or backward compatibility. Production protocols require a version negotiation mechanism that allows multiple versions to coexist during upgrade periods, with explicit downgrade protection to prevent attackers from forcing connections to use older, potentially less secure protocol versions.

9.7 Replay Window and Reordering

The current replay window size of 64 is appropriate for TCP, which guarantees in-order delivery at the transport layer. Any reordering that occurs at the IP layer is transparently corrected by TCP before the application sees the data. However, if the record layer were adapted for use over UDP (for example, to support QUIC-like protocols or real-time applications where TCP head-of-line blocking is unacceptable), the window size of 64 would need to be increased significantly, and the window management logic would need to handle larger gaps between received sequence numbers.

10. Conclusion

This report has presented the complete design, implementation, and evaluation of a post-quantum secure client-server protocol system. The system implements a full TLS-inspired handshake and record layer protocol in three operating modes — PQ_ONLY, CLASSICAL, and HYBRID — enabling direct, controlled comparisons of classical and post-quantum cryptographic approaches on identical hardware and software configurations.

The experimental results clearly show that post-quantum handshakes are practical on modern hardware. The default configuration (Kyber-768 + Dilithium-3, PQ_ONLY mode) achieves a median handshake latency of 122 ms — approximately 3.5 times higher than the classical baseline (34 ms), but well within acceptable bounds for connection establishment in most application scenarios. Connections are established once per session or per long-lived connection, so a 90 ms increase in connection setup time does not meaningfully affect the throughput or responsiveness of applications that exchange many records after the handshake.

The HYBRID mode adds negligible overhead compared to PQ_ONLY (median latency 119.6 ms vs. 122.3 ms, a difference that is not statistically significant) while providing defense in depth: the channel remains secure even if either X25519 or Kyber-768 is eventually broken. This makes HYBRID the recommended configuration for organizations that are beginning the transition to post-quantum cryptography and want to maintain both quantum resistance and classical security guarantees simultaneously.

The record layer performance is excellent across the full range of payload sizes: AES-256-GCM achieves 2,768 MB/s throughput at 64 KB payload sizes, with a constant per-record overhead of only 21 bytes (16-byte GCM tag + 5-byte framing). Per-record CPU time stays below 23

microseconds even at 64 KB, confirming that the record layer is not a bottleneck in any realistic application.

The security evaluation provides strong empirical evidence for the effectiveness of the protocol design. All six attack types (replay, bit-flip, fake server, downgrade, eavesdrop, and MITM) were detected with 100% detection rate and 0% success rate across all 18 mode-attack combinations, with zero false positives. The combination of canonical transcript binding, certificate validation with date enforcement, identity binding, stateless DoS cookies, IPsec-style replay protection, and fail-closed record processing provides comprehensive defense in depth against the threat model.

The parameter-level comparisons provide a clear practical conclusion: in a Python-based implementation, the performance difference between Dilithium2 and Dilithium3 (6.5 ms median difference, not statistically significant) and between Kyber512 and Kyber768 (no advantage for the smaller parameter set, with higher variance) both strongly support preferring the higher security level in every case. The higher security level costs nothing in practice, because the dominant overhead comes from Python interpreter and network round-trips, not from the lattice arithmetic that differs between parameter sets.

The remaining limitations — single-CA trust model, absence of automated revocation, no side-channel analysis, no fuzzing coverage, and single protocol version support — are acknowledged gaps between this research prototype and a production-grade deployment. Each limitation is well-understood and has a known mitigation path for future work. Within its stated scope, the system successfully demonstrates that post-quantum secure communications are achievable today with acceptable performance trade-offs, using NIST-standardized algorithms that are already available in mature open-source libraries.

References

[1] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-Quantum Authentication in TLS 1.3: A Performance Study," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2020.

[2] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking Post-Quantum Cryptography in TLS," in Proceedings of the Post-Quantum Cryptography Conference (PQCrypto), pp. 72–91, Springer, 2020.

[3] K. Kwiatkowski and A. Langley, "Measuring TLS key exchange with post-quantum KEM," in Workshop Record of the Cryptographic Hardware and Embedded Systems (TCHES), 2019.

[4] E. Crockett, C. Paquin, and D. Stebila, "Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH," IACR Cryptology ePrint Archive, Report 2019/858, 2019. [Online]. Available: https://eprint.iacr.org/2019/858

[5] National Institute of Standards and Technology (NIST), "Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," FIPS 203, August 2024. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final

[6] National Institute of Standards and Technology (NIST), "Module-Lattice-Based Digital Signature Standard (ML-DSA)," FIPS 204, August 2024. [Online]. Available: https://csrc.nist.gov/pubs/fips/204/final

[7] National Institute of Standards and Technology (NIST), "Post-Quantum Cryptography Standardization," NIST IR 8413, September 2022. [Online]. Available: https://csrc.nist.gov/publications/detail/nistir/8413/final

[8] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle, "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," in IEEE European Symposium on Security and Privacy (EuroS&P), pp. 353–367, 2018.

[9] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehle, "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme," IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES), vol. 2018, no. 1, pp. 238–268, 2018.

[10] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force (IETF), August 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

[11] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid key exchange in TLS 1.3," IETF Internet-Draft, draft-ietf-tls-hybrid-design, 2023. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/

[12] D. Bernstein, T. Lange, and R. Niederhagen, "Dual EC: A Standardized Back Door," in The New Codebreakers, Lecture Notes in Computer Science, vol. 9100, Springer, 2016. — Referenced for context on classical cryptography limitations.

[13] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, November 2007. [Online]. Available: https://csrc.nist.gov/publications/detail/sp/800-38d/final

[14] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, Internet Engineering Task Force (IETF), May 2010. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5869

[15] P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," SIAM Journal on Computing, vol. 26, no. 5, pp. 1484–1509, 1997.

[16] L. Grover, "A Fast Quantum Mechanical Algorithm for Database Search," in Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212–219, 1996.

[17] S. Kent and K. Seo, "Security Architecture for the Internet Protocol," RFC 4301, Internet Engineering Task Force (IETF), December 2005. — Referenced for the IPsec-style sliding window replay protection mechanism. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4301

[18] D. Stebila and M. Mosca, "Post-Quantum Key Exchange for the Internet and the Open Quantum Safe Project," in Selected Areas in Cryptography (SAC), Lecture Notes in Computer Science, vol. 10532, pp. 14–37, Springer, 2016.

[19] Open Quantum Safe Project, "liboqs: C Library for Quantum-Safe Cryptographic Algorithms," Version 0.15.0, 2024. [Online]. Available: https://openquantumsafe.org